

Angular : signal() VS signalState()

 Piyali Das 4 min read · 2 days ago

 50













This is one of the most important concepts when learning **NgRx Signal Store**.

TS

app.component.ts

×

...



```
1 import { Component, computed, signal } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { patchState, signalState } from '@ngrx/signals';
4 import { Person } from '../models/person.model';
5
6 @Component({
7   selector: 'app-root',
8   templateUrl: './app.component.html',
9   styleUrls: ['./app.component.scss'],
10  standalone: true,
11  imports: [CommonModule],
12 })
13 export class AppComponent {
14   readonly isShow = signal<Boolean>(false);
15 }
```

Fork on  StackBlitz



Unable to run Embedded Project

Looks like this project is being embedded without proper isolation headers.

[Read more about browser support for embedded projects](#)

Editor Preview Both

Feature	`signal()`	`signalState()`
Comes From	Angular	NgRx

Purpose	Single reactive value	Reactive object state
Nested Reactivity	No	Yes (Deep Signals)
Updates	<code>`set()`, `update()`</code>	<code>`patchState()`</code>
Computed Nested Properties	Manual	Automatic
Best For	Local UI state	Feature/Application state
Deep Property Tracking	✗	✓

Scenario	signal()	signalState()
-----	-----	-----
Change `name`	Entire object changes	Only `name` signal changes
Change `street`	Entire object changes	Only `street` signal changes
Computed Recalculation	More frequent	More targeted
Memory Usage	Lower	Slightly higher
Large State Objects	Less efficient	More efficient
Fine-Grained Reactivity	✗	✓

Signal vs SignalState — Update & Computation

Feature	<code>`signal()`</code>	<code>`signalState()`</code>
-----	-----	-----
Package	<code>`@angular/core`</code>	<code>`@ngrx/signals`</code>
State Type	Single value or object	Structured application state
Nested Reactivity	✗ Whole object tracked	✓ Deep Signals
Read Value	<code>`person()`</code>	<code>`person.name()`</code>
Update Value	<code>`set()`, `update()`</code>	<code>`patchState()`</code>
Computed Value	<code>`computed()`</code>	<code>`computed()`</code>
Nested Property Access	<code>`person().address.street`</code>	<code>`person.address.street()`</code>
Update Nested Property	Manual immutable update	<code>`patchState()`</code>
Best Use Case	UI state	Feature state

Example State

```
interface Person {  
  id: number;  
}
```

```

name: string;
address: {
  street: string;
  city: string;
};
}

```

1. State Creation

signal()

```

ts<br>readonly person = signal<Person>
({<br> id:1,<br> name:'John',<br> address:{
street:'Main St', city:'NY'}<br>});

```

signalState()

```

ts<br>readonly person = signalState<Person>
({<br> id:1,<br> name:'John',<br> address:{
street:'Main St', city:'NY'}<br>});

```

2. Reading Values

signal()	signalState()
<code>`ts<br>person().name`</code>	<code>`ts<br>person.name()`</code>
<code>`ts<br>person().address.street`</code>	<code>`ts<br>person.address.street()`</code>

Dependency Tracking

```

signal()
-----
person()
  ↓
whole object dependency

```

```

signalState()
-----

```

```
person.address.street()  
      ↓  
only street dependency
```

3. Updating Root Property

Change Name

signal()

```
this.person.update(person => ({  
  ...person,  
  name: 'Bob'  
}));
```

signalState()

```
patchState(this.person, {  
  name: 'Bob'  
});
```

Comparison	signal()	signalState()
Lines of Code	More	Less
Immutable Update Required	Yes	No
Readability	Medium	High

5. Computed Values

Full Address

signal()

```
readonly fullAddress = computed(  
  () =>  
    `${this.person().address.street},  
    ${this.person().address.city}`  
);
```

signalState()

```
readonly fullAddress = computed(  
  () =>  
    `${this.person.address.street()},  
    ${this.person.address.city()}`  
);
```

1. Angular signal()

```
readonly count = signal(0);
```

Read:

```
this.count()
```

Update:

```
this.count.set(10);
```

or

```
this.count.update(v => v + 1);
```

Object with signal

```
readonly person = signal<Person>({  
  id: 1,  
  name: 'John',  
  address: {  
    street: 'Main St'  
  }  
});
```

Access:

```
this.person().address.street
```

Notice:

```
person()
```

returns the entire object.

Angular treats it as a single signal.

```
person  
|  
+-- object
```

No separate signal exists for:

```
address  
street  
city
```

Updating a nested property

Must update immutably:

```
this.person.update(person => ({  
  ...person,  
  address: {  
    ...person.address,  
    street: 'New Street'  
  }  
}));
```

2. NgRx signalState()

```
readonly person = signalState<Person>({  
  id: 1,  
  name: 'John',  
  address: {  
    street: 'Main St'  
  }  
});
```

Now NgRx creates **Deep Signals**.

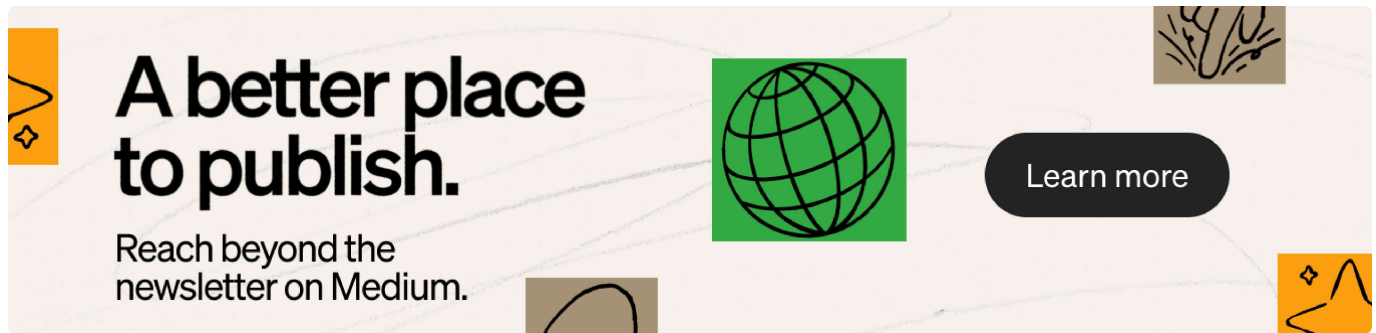
Access:

```
this.person.address.street()
```

Notice:

```
street()
```

because street itself is a signal.



Internally:

```
person
├── id
├── name
└── address
    └── street
```

Each property becomes reactive.

signal()

```
person (Signal<Person>)
|
+----- entire object
```

To get street:

```
this.person().address.street
```

signalState()

```
person
├─ id()
├─ name()
└─ address
    └─ street()
```

To get street:

```
this.person.address.street()
```

Why signalState is Powerful

Suppose:

```
readonly person = signalState({
  id: 1,
  name: 'John',
  address: {
    street: 'Main St',
    city: 'NY'
  }
});
```

Template:

```
{{ person.address.street() }}
```

Only the street signal is tracked.

```
street changes
  ↓
only street consumers update
```

With regular signal:

```
{{ person().address.street }}
```

Angular tracks:

```
person signal
```

The whole object dependency.

Updating State

signal()

```
this.person.update(person => ({
  ...person,
```

```
    name: 'Bob'  
  }));
```

signalState()

```
patchState(this.person, {  
  name: 'Bob'  
});
```

Cleaner.

Real Example

signal()

```
readonly filters = signal({  
  search: '',  
  page: 1,  
  pageSize: 20  
});
```

Read:

```
this.filters().search
```

Update:

```
this.filters.update(f => ({
  ...f,
  search: 'Angular'
}));
```

signalState()

```
readonly filters = signalState({
  search: '',
  page: 1,
  pageSize: 20
});
```

Read:

```
this.filters.search()
```

Update:

```
patchState(this.filters, {
  search: 'Angular'
});
```

Much cleaner.

When to Use `signal()`

Use for simple local state:

```
loading  
selectedTab  
theme  
dialogOpen  
counter  
searchText
```

Example:

```
readonly loading = signal(false);  
readonly selectedTab = signal(0);
```

When to Use `signalState()`

Use for complex object state:

```
User Profile  
Dashboard  
Shopping Cart  
Orders  
Products  
Feature State
```

Example:

```
readonly userState = signalState({  
  user: null,  
  loading: false,  
  error: null  
});
```

In Modern Angular 19/20

A common pattern is:

```
loading = signal(false);  
filters = signalState({  
  search: '',  
  page: 1  
});  
usersStore = inject(UsersStore);
```

Where:

- `signal()` → simple UI flags
- `signalState()` → complex feature state
- `signalStore()` → full-fledged state management with methods, computed values, and RxJS integration

Think of it as:

```
signal()  
  ↓  
single value
```

```
signalState()  
  ↓  
state object with deep signals  
  
signalStore()  
  ↓  
complete state management solution
```

That's why most NgRx Signal Store examples internally build on top of **signalState**, because it provides deep reactive state out of the box.

Angular +

UI +

Programming +

Technology +

Coding +

**Written by Piyali Das**

807 followers · 9 following

[Edit profile](#)

12 + yrs | Angular | NGRX | RxJS | <https://www.linkedin.com/in/piyalidas10/>
<https://stackblitz.com/@piyalidas10> | <https://github.com/piyalidas10>

